

Assignment 3 | Bridge Pattern

Topic A - Drawing

Student	[YOUR NAME]	Group	[YOUR GROUP]
Repository	[ADD YOUR GITHUB REPOSITORY URL]	Language	Java, JDK 17
Base commit	999ebe8b330d	Source commit	c2f9bed9d827

1. Design rationale

The program has two independent dimensions. The first is **what is drawn**: Shape, Circle, and Square. The second is **how it is rendered**: Renderer, VectorRenderer, RasterRenderer, and AsciiRenderer. Bridge fits because Shape stores a reference of type Renderer and the concrete shapes delegate work through that interface. This avoids classes such as VectorCircle or RasterSquare and lets the renderer change at runtime.

The code is intentionally small: each class has one clear responsibility, the fixed values are radius 2 and side 3, and all results are plain strings.

2. UML class diagram

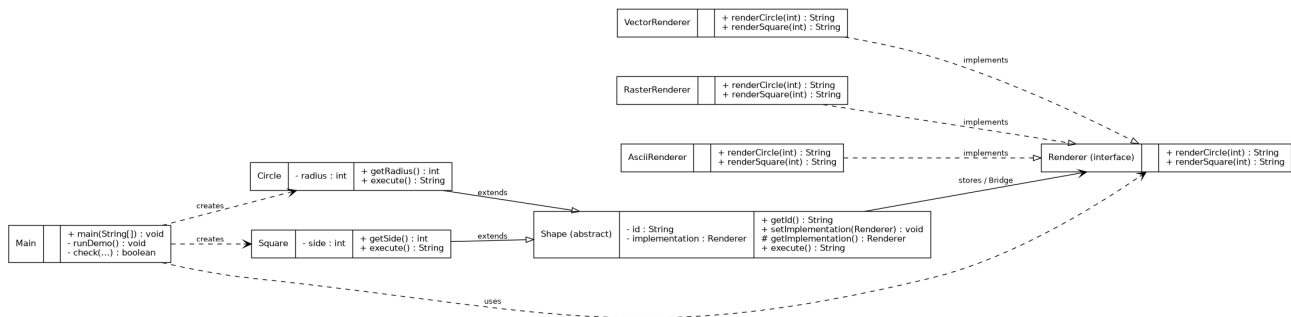


Figure 1. Shape contains the interface-typed bridge to Renderer. Circle and Square are refined abstractions. VectorRenderer, RasterRenderer, and AsciiRenderer implement the Renderer interface. Main is the client.

Role	Class(es)	Responsibility
Abstraction	Shape	Stores ID and Renderer reference; exposes execute() and setImplementation().
Refined abstractions	Circle, Square	Store radius/side and delegate to Renderer.
Implementor	Renderer	Declares renderCircle() and renderSquare().
Concrete implementors	VectorRenderer, RasterRenderer, AsciiRenderer	Return different rendering descriptions.
Client	Main	Runs T1-T7 and calculates PASS/FAIL.

3. Five annotated Clean Code excerpts

These examples are taken from the submitted source and use simple code that is easy to trace during the defense.

1) Separation of responsibilities - Shape depends on the interface

```
public abstract class Shape {
    private final String id;
    private Renderer implementation;
    ...
    public abstract String execute();
}
```

Benefit: Shape keeps general shape state and the Renderer reference. Rendering details stay inside renderer classes.

2) Meaningful names - Circle stores its own domain data

```
public class Circle extends Shape {
    private final int radius;

    public int getRadius() {
        return radius;
    }
}
```

Benefit: Names such as Circle, radius, Renderer, and setImplementation clearly describe the purpose of the code.

3) Small focused method - one delegation step

```
@Override
public String execute() {
    return getImplementation().renderCircle(radius);
}
```

Benefit: execute() has one job: pass Circle data to the current renderer and return the result.

4) Avoid duplicated checking logic - shared helper

```
private static boolean check(String testId, String classes,
    String actual, String expected) {
    boolean passed = actual.equals(expected);
    ...
    return passed;
}
```

Benefit: T1-T4 and T6-T7 use the same comparison method, so PASS/FAIL is calculated consistently instead of being hard-coded.

5) Encapsulated state - stable data, replaceable implementation

```
private final String id;
private Renderer implementation;

public void setImplementation(Renderer implementation) {
    this.implementation = implementation;
}
```

Benefit: The ID remains fixed while the renderer reference can change. T5 changes behavior without creating a new Circle or losing its data.

4. Runtime switch and independent extension

T5 creates one Circle with VectorRenderer. A reference is saved before the switch. Then the program calls **setImplementation(raster)** and executes the same Circle again. The check uses **==** to prove that the references point to the same object. It also checks that the ID and radius are unchanged. The output changes from VECTOR to RASTER because only the renderer reference changed.

The base commit is **999ebe8b330d4e680e932abfd265ca94443dbdbe**. The I3 source commit is **c2f9bed9d8275b0ad03db8674dec22a97314156a**. The extension adds AsciiRenderer and T6-T7 in Main. The existing Shape, Circle, Square, Renderer, VectorRenderer, and RasterRenderer source files are unchanged. The file extension.diff records this source change.

5. Bridge compared with Adapter

Aspect	Bridge in this project	Adapter
Intent	Separate abstraction from implementation so both can vary independently.	Make an existing incompatible interface usable by a client.
When used	Designed into the program from the start.	Usually added when incompatible existing interfaces must work together.
Here	Shape delegates to interchangeable Renderer objects.	No legacy incompatible renderer is being converted.

6. Trade-off and verification

One trade-off of Bridge is that it adds an interface, extra classes, and one level of delegation. That would be unnecessary for a tiny program with only one fixed shape and one fixed renderer. Here it is useful because both sides vary and the renderer must be replaceable at runtime.

The program was compiled with the required JDK 17 command and executed with `Main --demo`. The captured result is **SUMMARY: 7/7 PASS**. T1-T4 test the complete two-by-two matrix, T5 tests runtime replacement on the same object, and T6-T7 test AsciiRenderer.

7. References

1. Almas Ospanov. *Lecture 4 - The Structural Design Pattern: Bridge Pattern*. Course lecture slides.
2. Yerassyl Bekenov. *Assignment 3 | Bridge Pattern*. ShP-2216 Software Design Patterns, 2026-2027.

Submission note: replace [YOUR NAME], [YOUR GROUP], and the repository URL before submission.